Expressivity Engineering for Single-Layer Reinforcement Learning

A Controlled TriGLU Study on Qwen3-1.7B-Base

Hanting Wu
Independent Research

July 2026

Abstract

We introduce *Expressivity Engineering* (EE), a design perspective that studies how higher-order, multiplicative, and input-conditioned interactions can be added to neural network computation beyond pure linear channel mixing. To test this proposal under a bounded compute budget, we use the layer-selective reinforcement learning workflow of Zhang et al. [2026] as experimental guide rails: Qwen3-1.7B-Base is trained with Group Relative Policy Optimization while only decoder Layer 10 is updated. The EE intervention retains the original SwiGLU path and adds an exact-no-op-initialized, FP32 side network that multiplicatively modulates the Layer-10 SwiGLU hidden state. The implementation, called TriGLU in code and more descriptively ToTGLU in this report, adds 20,986,368 parameters, or 41.69% of one Qwen3-1.7B decoder layer.

We compare the intervention with a matched naive whole-Layer-10 baseline. Both runs start from the same untuned Qwen3-1.7B-Base revision and share data order, initialization and rollout seeds, reward, response cap, and a six-GPU `504/126/6` train/mini/micro-batch contract. Corrected evaluations are reported at matched global steps 158, 196, 226, 256, and 294. Averaged across these correlated checkpoints, TriGLU improves MathAvg from 52.224 to 52.997 (+0.773 points) and the provisional CodeAvg from 32.671 to 33.659 (+0.988), while Reasoning and Language category averages are approximately tied (-0.213 and -0.128). The clearest gains occur on harder generative tasks: OlympiadBench (+2.548), corrected LiveCodeBench (+1.063), and a manually audited GPQA-Diamond-Freeform diagnostic (+1.746). These results are promising but not definitive: they come from one training seed, checkpoint variation is not independent-seed uncertainty, several evaluation protocols required correction, and response-length caps remain in both training and evaluation. The study therefore establishes an artifact-backed positive pilot for EE-PEFT, not a general scaling-law claim.

1 Introduction

Modern Transformer blocks alternate token mixing with channel mixing. Within the feed-forward network (FFN), a linear map can mix channels but cannot by itself express multiplicative interactions among features. Gated linear units already weaken this limitation by multiplying two learned streams; SwiGLU is a particularly successful example [Shazeer, 2020]. Hypernetworks generate or modulate parameters from context [Ha et al., 2016]; mixtures of experts route inputs through specialized computation; polynomial activations explicitly introduce higher-order terms. These methods are usually discussed in separate literatures despite sharing a practical objective: making the model's effective computation depend more richly on its current state.

We propose *Expressivity Engineering* as a unifying engineering lens for that objective. In this report, an EE operation is one that introduces a learned multiplicative, higher-order, routed, or dynamically parameterized interaction, as opposed to only applying another static linear channel-mixing map. This is a working taxonomy rather than a theorem that totally orders neural architectures by expressive power. It is useful because it separates two questions:

1. How much additional input-dependent interaction is introduced?
2. Where can that interaction be inserted and trained economically?

The present study addresses the second question through layer-selective RL. Zhang et al. [2026] show that RL gains in Qwen3 and related models are concentrated in a small set of middle layers; on Qwen3-1.7B-Base, Layer 10 is their strongest single-layer math result. That finding is not the conceptual origin of EE. It supplies the empirical localization result and workflow guide rails that make a compute-bounded test of our independently proposed EE hypothesis feasible: instead of pretraining a new architecture from scratch, we can add expressivity at one high-contribution layer and compare it against a naive update of exactly the same layer.

Our contributions are:

- a precise EE taxonomy connecting multiplicative FFNs, hypernetworks, routed low-rank experts, grid-wise modulation, and polynomial activations;
- an exact-function-preserving Layer-10 TriGLU/ToTGLU intervention for Qwen3-1.7B-Base;
- a matched six-GPU GRPO comparison from the same untuned base model, with deterministic data exposure and five common evaluation checkpoints;
- corrected Math and out-of-distribution evaluation records, including a strict free-form GPQA-Diamond audit and explicit protocol/cap-hit boundaries;
- a systems account of registered Hugging Face/vLLM integration, continuous batching, live weight synchronization, and the current serving-performance cost of the FP32 side branch.

2 Expressivity Engineering

2.1 A working definition

For an input vector x, a static linear map Wx performs channel mixing. EE introduces interactions such as

$$y = Wx \odot g(x), \qquad y = (W + \Delta W(x))\, x, \qquad y = \sum_e \pi_e(x) f_e(x), \tag{1}$$

where the effective transform depends on x, or where multiple learned streams are multiplied or routed. Under this definition:

- GLU/SwiGLU uses multiplicative feature interactions [Shazeer, 2020];
- HyperNetworks generate parameters for another network [Ha et al., 2016];
- HyperGrid decomposes task-conditioned modulation into grid-wise projections [Tay et al., 2020];
- HyperFormer generates task/layer/position-conditioned adapter parameters, and HyperLoRA generates constrained low-rank adapters [Mahabadi et al., 2021, Lv et al., 2024];
- MoLoRA treats lightweight low-rank adapters as routed experts [Zadouri et al., 2024];
- PolyCom/PolyNorm makes higher-order interactions explicit in the activation family [Zhuo et al., 2024].

These methods are related, not interchangeable. HyperFormer and HyperLoRA are primarily task-conditioned parameter generators; MoLoRA routes among learned lightweight experts; HyperGrid modulates structured regions of weight matrices; PolyNorm uses elementwise polynomial powers and normalization. The present TriGLU branch instead computes a per-token multiplicative modulation of one existing FFN hidden state.

2.2 Scope beyond Transformers

The EE lens is not limited to language models. Any architecture containing an MLP can, in principle, be augmented with multiplicative, routed, polynomial, or dynamically parameterized computation. For Transformer FFNs, such operations remain linear in sequence length, unlike dense self-attention's quadratic token-pair interaction. This does not make EE free: added FFN branches increase FLOPs, activation memory, kernel-launch overhead, and optimization difficulty. It does mean that richer channel computation can be explored without directly increasing the number of attention score pairs or the KV-cache length.

Attention is itself an input-conditioned token-mixing mechanism and can be described within the broader EE taxonomy. This controlled study nevertheless keeps the attention architecture unchanged: only the Layer-10 FFN computation differs between conditions, while the ordinary Layer-10 attention parameters remain trainable in both.

3 Model

3.1 Backbone and update scope

The backbone is Qwen3-1.7B-Base [Yang et al., 2025]: 28 decoder layers, hidden width 2,048, FFN intermediate width 6,144, 16 query heads, 8 key/value heads, and a 32,768-token context window. Both the baseline and EE

variant update decoder Layer 10 only. Layer-10 attention projections, RMSNorm parameters, and all original FFN projections are trainable. Embeddings, the language-model head, and all other decoder layers are frozen.

This is a stricter comparison than using the untuned base model as the primary control. The question is not whether RL helps, but whether adding a particular expressivity mechanism improves a strong naive single-layer RL update.

3.2 TriGLU/ToTGLU side modulation

Let $x \in \mathbb{R}^{2048}$ be the Layer-10 FFN input. The original SwiGLU hidden state is

$$h_0 = \mathrm{SiLU}(W_g x) \odot (W_u x), \qquad h_0 \in \mathbb{R}^{6144}. \tag{2}$$

The FP32 side network first compresses to a 512-dimensional bottleneck, $z = Ax \in \mathbb{R}^{512}$, and forms three 2,048-dimensional streams:

$$v = Vz, \tag{3}$$
$$g = \mathrm{SiLU}(Gz), \tag{4}$$
$$q = \mathrm{SiLU}(F_2\,\mathrm{GeLU}(F_1 z))\,. \tag{5}$$

Their product is projected to the backbone intermediate width,

$$s = U(v \odot g \odot q) \in \mathbb{R}^{6144}, \tag{6}$$

and modulates the original hidden state:

$$h = h_0 \odot [1 + \alpha \tanh(s)]\,, \qquad y = W_d h, \qquad \alpha = 0.1. \tag{7}$$

The code calls this module TriGLU. Because the base SwiGLU already contains two multiplicative streams and the side branch multiplies three more, we use ToTGLU ("TriGLU on TriGLU") when emphasizing its five-stream interpretation. The name is descriptive rather than a claim that the module implements a standard architecture from prior work.

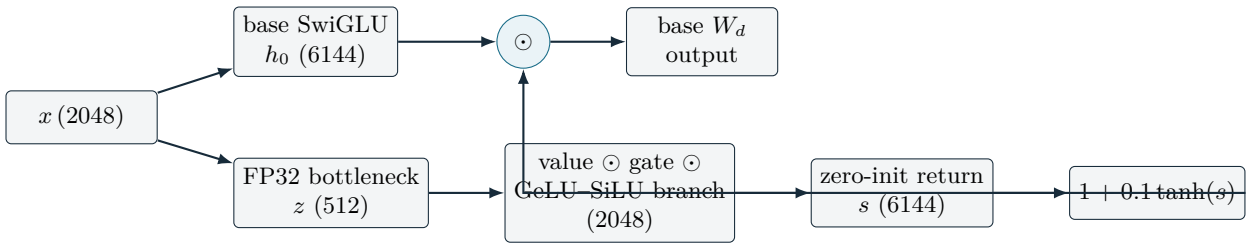

Figure 1: Layer-10 TriGLU/ToTGLU intervention. The original SwiGLU path is retained. A per-token FP32 side network produces a bounded multiplicative modulator before the original down projection.

3.3 Exact no-op initialization and parameter cost

The side return projection U (named `triglu_side.up` in code) is initialized to zero. Therefore $s = 0$, the multiplier in Eq. 7 is exactly one, and the initial model is functionally identical to the unmodified backbone. This invariant isolates subsequent behavior from a random architecture shock.

The active dimensions are `side_dim=512` and `side_hidden=2048`. The side network contains 20,986,368 trainable parameters, 41.69% of the 50,337,792-parameter selected decoder layer. The base Layer-10 parameters remain jointly trainable. This is parameter-efficient relative to full-model RL, but it is deliberately a high-capacity intervention rather than a minimal adapter.

Table 1: Six-rank production contract.

Field	Value
Train batch	504 prompts
PPO mini batch	126 prompts
PPO micro batch	6 prompts
Group size	4 responses/prompt
Responses per global update	2,016
Prompts through step 98	49,392
Deterministically omitted at step 98	608 / 50,000
Rollout replicas	6 independent TP=1 replicas

4 Experimental Design

4.1 Relationship to the layer-selective RL workflow

The experiment follows the operational structure of Zhang et al. [2026]: Qwen3-1.7B-Base, Layer 10, NuminaMath-CoT, GRPO, single-layer freezing, a 3,072-token response cap, and Math plus OOD benchmark families. The original work reports that Layer 10 reaches 51.8 MathAvg versus 50.8 for full-parameter RL under its protocol. Its results and hyperparameters are guide rails, not copied output: its executable repository and exact evaluation harness were not available to this study, so absolute score parity is not claimed.

Our scientific variable is the Layer-10 FFN architecture. The baseline updates the ordinary Layer-10 Transformer block; the EE condition updates that same block plus the side network. Both begin from the same untuned base model.

4.2 Data and decontamination

Training uses a 50,000-problem materialization derived from NuminaMath-CoT [Li et al., 2024]. A decontamination pass removes question-level overlap with the evaluation sets before training. The two variants use the same parquet hashes, deterministic shuffled permutation, omitted-row set, initialization seed, and rollout seed (`20260707`). This controls data exposure within the comparison; it does not prove that the local data processing exactly matches an unavailable external implementation.

4.3 GRPO and the six-GPU batch contract

GRPO samples a group of responses per prompt and normalizes their advantages within the group, avoiding a learned critic [Shao et al., 2024]. The implementation uses veRL/HybridFlow [Sheng et al., 2025] with vLLM rollouts [Kwon et al., 2023]. The paper-aligned tuple is train/mini/micro batch `512/128/8`, group size 4, KL coefficient 0.001, clip range 0.2, and four epochs. The actual six-RTX-5090 matched run uses the explicitly approved divisible tuple in Table 1.

The loader uses `drop_last=True`; step 98 is therefore a *near-one-pass* milestone, not an exact epoch. No irregular tail batch is introduced. Both architectures omit exactly the same 608 rows, giving a matched comparison with a 1.56% exposure deviation from 512×98.

4.4 Training stages and post-hoc interventions

The trajectory contains three 98-update stages. The first uses the original 5×10^{-6} learning rate. The second keeps that rate through step 128 and then applies cosine decay to 5×10^{-7} at step 196. The third decays from 5×10^{-7} to 5×10^{-8} at step 294. Each variant's frozen KL reference is reset to its own step-98 policy for updates 99–196 and to its own step-196 policy for updates 197–294.

The decay and reference resets were introduced during the live research cycle, not preregistered before update 1. Step 158 is the first reported common checkpoint after the step-128 schedule change and the first point where the

architectural trajectories visibly separated. We report all available matched major checkpoints from 158 onward rather than selecting a single best model.

4.5 Historical SFT pilot

Before the main RL wave, a two-epoch, 50K-example supervised fine-tuning pilot compared SHS, the whole-layer baseline, TriGLU, and OFT. It completed operationally, but did not provide a reliable architecture selection signal. SHS, baseline, and TriGLU had four-task Math averages within 0.32 points. OFT achieved the lowest teacher-forced validation loss (0.5893) yet collapsed under autoregressive evaluation (13.04 Math average versus approximately 43 for the other three). This is direct evidence in this project that same-distribution SFT loss is not a sufficient proxy for the intended generative reasoning behavior. Consequently, the main GRPO comparison starts from the untuned base, not from an SFT checkpoint.

OFT [Qiu et al., 2023] was scaffolded but not run as a GRPO comparator. The implemented policy freezes the original Layer-10 SwiGLU matrices and trains orthogonal rotations, whereas TriGLU jointly trains the original SwiGLU and the side branch. A direct comparison would therefore confound architectural expressivity with different backbone adaptation constraints. OFT remains a useful future control under a matched update policy.

5 Evaluation

5.1 Benchmark suite and aggregate definitions

The in-domain Math category follows MATH-500 [Hendrycks et al., 2021], GSM8K [Cobbe et al., 2021], Olympiad-Bench [He et al., 2024], and the 2023 AMC 10/12 competition set [Mathematical Association of America, 2023]. MathAvg is the unweighted mean of those four scores and uses AMC Average@32, not AMC greedy pass@1. AMC greedy is retained as a modal-path diagnostic.

OOD evaluation includes:

- Code: corrected HumanEval+ [Liu et al., 2023], provisional MBPP [Austin et al., 2021], and corrected Live-CodeBench [Jain et al., 2024];
- Reasoning: GPQA-Diamond [Rein et al., 2023] and MMLU-Pro [Wang et al., 2024], plus a separate GPQA-Diamond-Freeform manual diagnostic that is not included in ReasoningAvg;
- Language and instruction following: C-Eval [Huang et al., 2023], IFEval [Zhou et al., 2023], and MGSM [Shi et al., 2022].

Except AMC Average@32, the primary corrected benchmark cells use greedy decoding. This choice is especially important for the FP32 TriGLU model, whose sampled generations at temperature 1 were less stable. The report does not convert that observation into a deployment recommendation; temperature and logit-distillation behavior require a dedicated study.

5.2 Benchmark semantics and relative difficulty

The suite intentionally spans different capability regimes. "Difficulty" below is a qualitative task-level description, not a claim that scores are directly comparable across datasets.

Table 2: Benchmark roles in the evaluation suite.

Benchmark	Task and answer format	Relative regime
MATH-500	Competition-style secondary mathematics; free-form numeric or symbolic answer	Harder than routine school arithmetic; broad algebra, geometry, number theory, and combinatorics
GSM8K	Grade-school arithmetic word problems; free-form short answer	Elementary mathematics with multi-step linguistic reasoning
OlympiadBench	Olympiad-oriented mathematics; free-form derivation and answer	Hardest in-domain mathematical set in this report
AMC 2023	AMC 10/12 high-school contest questions; multiple choice	Between routine K–12 work and olympiad-style proof problems
HumanEval+	Function synthesis from a specification; execution-based tests	Compact code generation with strengthened hidden tests
MBPP	Short natural-language programming tasks; execution-based tests	Introductory-to-intermediate code generation; score remains provisional here
LiveCodeBench	Time-indexed competitive-programming problems; execution-based tests	Harder and less contamination-prone coding evaluation
GPQA-Diamond	Graduate-level science questions; four-way multiple choice	Expert factual and scientific reasoning; 25% random-choice floor
MMLU-Pro	Ten-choice, multi-domain knowledge and reasoning	Broad professional/academic reasoning with a 10% random-choice floor
GPQA-Diamond-Freeform	Same science questions without answer options; manual semantic audit	Stricter diagnostic that removes the multiple-choice floor
C-Eval	Chinese multi-subject multiple choice	Broad school-to-professional knowledge in Chinese
IFEval	Verifiable instruction-following constraints	Tests compliance rather than subject-matter depth
MGSM	Multilingual grade-school mathematics; free-form short answer	Elementary mathematics under cross-lingual generation pressure

5.3 Protocol corrections

Several legacy evaluation cells measured harness failures rather than model capability. The corrected table therefore uses:

- a raw no-chat HumanEval+ route after the chat-wrapped protocol caused Hebrew/template collapse in both TriGLU and the untuned base;
- a corrected LiveCodeBench sandbox output contract after the legacy contract produced an artificial all-zero result;
- explicit separation of AMC Average@32 from greedy pass@1;
- a row-by-row semantic audit of 1,260 GPQA-Diamond-Freeform responses, with uncertain judgments counted as incorrect.

These corrections preserve prompts and scoring intent, but do not establish paper-exact evaluation parity. MBPP still lacks a full protocol-matrix rerun; its score and the derived CodeAvg remain provisional.

5.4 Response-cap boundary

All Math benchmarks contain 3,072-token cap hits: 5.02% on MATH-500, 0.43% on GSM8K, 13.85% on OlympiadBench, 3.84% on AMC Average@32, and 8.25% on AMC greedy. Training rollouts also contain nonzero cap hits. Many inspected cases are repetition loops, but a minority could conceivably reach a correct answer if continued. We retain the paper-style cap-hit-inclusive results for matched comparison and do not describe them as uncapped capability ceilings.

OOD cap rates are more heterogeneous. HumanEval+ is 2.50%, LiveCodeBench 9.73%, GPQA-Diamond 9.85%, MMLU-Pro 4.73%, MGSM 8.61%, and GPQA-Diamond-Freeform 7.62%. MBPP (93.92%), C-Eval (68.31%), and

IFEval (39.09%) are severe exceptions that require protocol-aware reruns before strong absolute interpretation.

6 Results

6.1 Across-checkpoint summary

Table 3 reports population mean and standard deviation across the five matched checkpoints. The checkpoints lie on one trajectory and are strongly correlated; the standard deviation measures checkpoint variation, not independent-seed uncertainty or a confidence interval.

Table 3: Corrected matched-checkpoint results (steps 158/196/226/256/294). Values are mean ± population standard deviation in percentage points. *MBPP and therefore CodeAvg remain provisional.

Metric	TriGLU	Baseline	Mean difference
MATH-500	67.520 ± 1.230	66.880 ± 0.412	$+0.640$
GSM8K	82.942 ± 0.322	82.396 ± 0.446	$+0.546$
OlympiadBench	28.059 ± 1.294	25.511 ± 0.533	$+2.548$
AMC 2023 Average@32	33.469 ± 0.755	34.109 ± 0.950	-0.640
MathAvg	$\mathbf{52.997 \pm 0.184}$	$\mathbf{52.224 \pm 0.393}$	$\mathbf{+0.773}$
AMC 2023 greedy	38.000 ± 1.000	35.500 ± 2.915	$+2.500$
HumanEval+ corrected	60.854 ± 0.896	59.512 ± 1.131	$+1.342$
MBPP*	29.280 ± 1.121	28.720 ± 0.844	$+0.560$
LiveCodeBench corrected	10.845 ± 0.401	9.782 ± 0.139	$+1.063$
CodeAvg*	$\mathbf{33.659 \pm 0.260}$	$\mathbf{32.671 \pm 0.567}$	$\mathbf{+0.988}$
GPQA-Diamond (MCQ)	26.566 ± 1.880	26.565 ± 2.273	$+0.001$
MMLU-Pro	33.895 ± 0.576	34.320 ± 0.198	-0.425
ReasoningAvg	$\mathbf{30.230 \pm 0.691}$	$\mathbf{30.443 \pm 1.121}$	$\mathbf{-0.213}$
GPQA-Diamond-Freeform	6.349 ± 1.328	4.603 ± 1.053	$+1.746$
C-Eval	47.548 ± 0.566	46.182 ± 0.560	$+1.366$
IFEval	27.421 ± 0.669	26.752 ± 0.657	$+0.669$
MGSM	56.538 ± 5.340	58.960 ± 0.199	-2.422
LanguageAvg	$\mathbf{43.836 \pm 1.724}$	$\mathbf{43.964 \pm 0.334}$	$\mathbf{-0.128}$

6.2 What the result supports

The primary Math result is positive: TriGLU improves MathAvg by 0.773 points across the five matched checkpoints. The gain is not uniform. It is largest on OlympiadBench and is accompanied by positive MATH-500 and GSM8K differences, while AMC Average@32 is lower. The separate greedy AMC diagnostic favors TriGLU by 2.5 points, illustrating that sampled distribution-wide reliability and the greedy modal path need not move together.

The OOD profile is also structured rather than uniformly positive. Corrected HumanEval+ and LiveCodeBench favor TriGLU, with the largest relative coding gain on the harder LiveCodeBench task. GPQA-Diamond MCQ is effectively tied, MMLU-Pro slightly favors the baseline, and strict free-form GPQA favors TriGLU. C-Eval and IFEval improve, but a large MGSM deterioration at step 294 creates high TriGLU variance and removes the category-average gain. The source guide's intuition—extra expressivity may improve hard reasoning while occasionally destabilizing simpler knowledge retrieval or multilingual behavior—is consistent with this profile, but is not uniquely established by it.

Training reward and downstream evaluation were not monotonically aligned. At several checkpoints the EE variant reached equal or lower recent reward while obtaining stronger held-out metrics. This is evidence against selecting these models by training reward alone. It does not show that lower reward causes better generalization; reward sampling, shuffled-batch composition, KL reference resets, and schedule changes all vary along the same trajectory.

6.3 Why checkpoint averaging is used

From step 158 onward, both variants occupy an oscillatory plateau: different shuffled batches pull the model toward different capability profiles. In a production workflow, researchers often select a checkpoint during such a plateau. Averaging matched checkpoints therefore reduces dependence on one post-hoc checkpoint choice and describes the typical observed trajectory. However, five checkpoints from one run are not five independent experiments. No claim of statistical significance is made, and a multiple-seed replication is the most important missing validation.

6.4 Full corrected checkpoint table

Figure 2 provides the complete per-checkpoint record. The image is generated from the repository's machine-readable consolidated table; it excludes SFT, untuned-base, and early RL checkpoints by design.

The body contains ten matched cells: five checkpoints for TriGLU and five for the baseline. The final rows report the population mean and standard deviation across checkpoints within each trajectory. These are descriptive trajectory summaries rather than multi-seed confidence estimates.

All displayed values use the corrected protocol where a correction was available. HumanEval+ uses the raw no-chat route, and LiveCodeBench uses the repaired execution contract. MBPP and its dependent CodeAvg remain marked provisional because their full protocol matrix is incomplete. MathAvg is computed from MATH-500, GSM8K, OlympiadBench, and AMC Average@32; the adjacent AMC greedy column is intentionally outside that aggregate. Likewise, GPQA-Diamond-Freeform is shown beside ReasoningAvg but is excluded from it. The table retains response-cap-inclusive values, so it should be read together with Section 5.4 rather than as an uncapped capability ceiling.

Qwen3-1.7B RL: Corrected Math + OOD Results Across Checkpoints

Matched steps 158 / 196 / 226 / 256 / 294 | scores in percentage points | mean +/- population std

Setting	Math						Code				Reasoning				Language			
	MATH-500	GSM8K	OlympiadBench	AMC 2023 Avg@32	MathAvg	AMC 2023 greedy*	HumanEval+ corrected	MBPP*	LiveCodeBench corrected	CodeAvg*	GPQA-Diamond	MMLU-Pro	ReasAvg	GPQA-Diamond-Freeform*	C-Eval	IFEval	MGSM	LangAvg
TriGLU-158	69.200	82.942	26.815	33.359	53.079	40.000	59.146	30.401	10.995	33.514	26.766	33.991	30.378	5.556	46.582	28.513	59.417	44.837
TriGLU-196	66.800	82.714	28.296	33.672	52.871	37.500	61.585	28.601	10.807	33.664	26.768	34.259	30.513	4.762	47.399	26.834	57.564	43.932
TriGLU-226	66.200	82.638	29.778	33.828	53.111	37.500	60.976	30.599	10.903	34.159	25.254	34.101	29.677	5.556	47.548	26.624	60.146	44.773
TriGLU-256	68.800	83.548	26.370	32.109	52.707	37.500	61.585	27.600	11.376	33.521	24.242	34.353	29.297	7.937	48.218	27.673	59.566	45.152
TriGLU-294	66.600	82.866	29.037	34.375	53.219	37.500	60.976	29.200	10.142	33.439	29.800	32.770	31.285	7.937	47.990	27.460	46.000	40.483
TriGLU mean +/- std	**67.52+/-1.23**	**82.94+/-0.32**	**28.06+/-1.29**	**33.47+/-0.76**	**53.00+/-0.18**	**38.00+/-1.00**	**60.85+/-0.90**	**29.28+/-1.12**	**10.84+/-0.40**	**33.66+/-0.26**	**26.57+/-1.88**	**33.89+/-0.58**	**30.23+/-0.69**	**6.35+/-1.33**	**47.55+/-0.57**	**27.42+/-0.67**	**56.54+/-5.34**	**43.84+/-1.72**
Baseline-158	66.800	82.942	26.370	33.750	52.465	35.000	59.756	29.598	9.764	33.039	28.283	34.432	31.358	6.349	45.392	26.416	58.909	43.572
Baseline-196	67.200	82.032	24.889	32.656	51.694	37.500	60.976	28.202	9.764	32.980	22.727	34.533	28.630	4.762	45.692	27.046	59.017	43.918
Baseline-226	67.400	82.638	25.778	35.156	52.743	32.500	60.366	29.600	10.046	33.337	25.250	34.027	29.638	3.175	46.284	25.998	58.726	43.669
Baseline-256	66.800	81.729	25.037	33.828	51.848	40.000	57.927	28.800	9.667	32.131	27.778	34.466	31.122	3.968	46.731	27.884	58.838	44.484
Baseline-294	66.200	82.638	25.481	35.156	52.369	32.500	58.537	27.401	9.670	31.869	28.788	34.143	31.465	4.762	46.808	26.417	59.309	44.178
Baseline mean +/- std	**66.88+/-0.41**	**82.40+/-0.45**	**25.51+/-0.53**	**34.11+/-0.95**	**52.22+/-0.39**	**35.50+/-2.92**	**59.51+/-1.13**	**28.72+/-0.84**	**9.78+/-0.14**	**32.67+/-0.57**	**26.57+/-2.27**	**34.32+/-0.20**	**30.44+/-1.12**	**4.60+/-1.05**	**46.18+/-0.56**	**26.75+/-0.66**	**58.96+/-0.20**	**43.96+/-0.33**

MathAvg uses AMC 2023 Avg@32, not AMC 2023 greedy*. ReasAvg uses GPQA-Diamond + MMLU-Pro, not GPQA-Diamond-Freeform*.

Code note: MBPP is the current heritage/provisional score; the full protocol-matrix correction is pending, so CodeAvg* inherits this boundary.

Corrected routes: HumanEval+ raw no-chat + fixed parser/sandbox; LiveCodeBench fixed sandbox output contract; GPQA-F strict manual audit.

Math cap hits are retained: MATH500 5.02%, GSM8K 0.43%, Olympiad 13.85%, AMC@32 3.84%, AMC greedy 8.25%. Training rollouts also contain cap hits.

Many inspected cap hits are repetition loops, but a small number could reach a correct answer if continued; results are not uncapped capability ceilings.

Math difficulty (rough): GSM8K < AMC 2023 < MATH-500 / OlympiadBench. Avg@32 measures sampled-distribution reliability; greedy is modal pass@1.

Code difficulty (rough): MBPP < HumanEval+ < LiveCodeBench. Scores depend on executable-test and sandbox contracts.

Reasoning: MMLU-Pro is broad advanced MCQ; GPQA-Diamond is specialist graduate-science MCQ; GPQA-Diamond-Freeform is harder without choices.

Language: C-Eval measures Chinese multi-subject exams; IFEval measures constraint following; MGSM adds multilingual burden to grade-school math.

Figure 2: Full corrected Math and OOD results at matched RL steps 158, 196, 226, 256, and 294, including across-checkpoint means and population standard deviations. MathAvg uses AMC Average@32, not AMC greedy. ReasoningAvg excludes GPQA-Diamond-Freeform. MBPP and CodeAvg remain provisional.

7 Systems Integration and Efficiency

The study required an architecture-aware serving route rather than only a training-time PyTorch patch. The repository provides an explicit `Qwen3TriGLUForCausalLM`, an out-of-tree vLLM plugin, export metadata, dispatch receipts, post-load FP32 restoration for the side branch, live weight synchronization, and exact greedy-token parity in bounded HF/vLLM gates. The production custom math still executes as ordinary PyTorch/cuBLAS operations inside vLLM; no production TriGLU Triton kernel is claimed.

On two RTX 5090 GPUs, using independent TP=1 vLLM replicas, pressure-64 matched long decoding reached 2,872.1 generated tokens/s/GPU for TriGLU versus 5,442.1 for vanilla Qwen, or 52.8% of the vanilla rate. The same TriGLU path was 151.8% of the historical SHS reference path (1,892.1 tokens/s/GPU). Thus the architecture is substantially slower than the baseline even though it modifies only one layer: autoregressive decoding repeatedly invokes that layer, and the FP32 side path adds six projections, three nonlinear streams, elementwise products, and conversion/kernel-launch overhead at every generated token.

This result motivates, but does not yet validate, three optimizations: a pure BF16 architecture path, fused side-branch kernels, and a native registered serving implementation that minimizes framework transitions. Continuous batching is already demonstrated and is architecture-agnostic at the scheduler level; what must be integrated is the model's custom forward path, weight loading, export metadata, and synchronization semantics.

If backend sensitivity remains after strict parity work, a second deployment path is to use the high-capacity EE model as a teacher rather than the final serving artifact. Calibrated logit or sequence distillation into a conventional student could preserve part of the learned capability while recovering mature kernels and serving support. This is a fallback research direction, not an explanation of the present gains, and it requires its own temperature, calibration, and teacher–student fidelity study.

8 Limitations

1. **One seed.** Checkpoint averaging cannot substitute for independent training seeds.
2. **Live schedule changes.** Learning-rate decay and KL-reference resets were introduced during the run. They are matched between variants, but prevent a clean claim about the original paper schedule.
3. **Protocol divergence.** The study did not have an executable paper repository. Local absolute scores differ from the paper, and several OOD evaluators required protocol repairs.
4. **Cap hits.** Training and evaluation contain truncated generations. High-cap OOD cells are not capability ceilings.
5. **Provisional MBPP.** CodeAvg inherits MBPP's unresolved protocol boundary.
6. **Manual free-form audit.** GPQA-Diamond-Freeform provides a strict complementary diagnostic, but its semantic judgments include 13 uncertain responses counted as incorrect and are not directly interchangeable with MCQ accuracy.
7. **Capacity mismatch.** TriGLU adds 41.69% of one layer's parameters. The experiment tests this concrete architecture, not a parameter-matched universal EE principle.
8. **Precision and backend sensitivity.** The FP32 side branch and generic PyTorch/cuBLAS execution affect both throughput and numerical behavior. Pure-BF16 and strict HF/vLLM parity remain pending.

9 Future Work

9.1 Immediate controlled experiments

The next scientific priorities are:

- repeat the matched baseline/TriGLU comparison across multiple seeds;
- complete the HF/vLLM evaluation parity matrix and cap-repair program;
- train pure-BF16 TriGLU and SHS variants, with accuracy and throughput gates;
- compare lower EE levels by reducing side width or the number of multiplicative streams;
- run OFT only under a matched backbone-adaptation policy, separating geometry preservation from trainable-capacity differences;

- replace fixed GeLU components with trainable polynomial activations, informed by PolyCom/PolyNorm [Zhuo et al., 2024];
- test per-token HyperGrid- or HyperNetwork-style modulation, without sequence pooling and beginning with factorized forms rather than full dynamic weight generation.

9.2 Joint adaptation and expressivity matching

The current intervention assumes that frozen surrounding layers can consume a representation produced by a more expressive Layer 10. The MGSM instability suggests that this interface can become a bottleneck. Two extensions follow. First, train adjacent layers jointly, perhaps with full updates near Layer 10 and constrained updates farther away. Second, add progressively weaker EE modules to neighboring layers, analogous to impedance matching: rather than a single abrupt change in representational geometry, increase and decrease the available interaction order gradually across depth.

The layer-selective paper reports that guided multi-layer updates are especially strong on larger Qwen3 models, including a best-10 strategy on Qwen3-8B [Zhang et al., 2026]. A direct Qwen3-8B B10 EE-PEFT comparison is therefore a high-value follow-up. It would provide more layers in which the added architecture can co-adapt and test whether the present single-layer gains are limited by surrounding-layer geometry.

9.3 Fresh pretraining

The cleanest test of EE is fresh pretraining, where every layer can adapt to the new computation. It removes the need for a residual $(1 + \delta)$ constraint and permits side streams with scale comparable to the main stream. A compute-aware program would begin with a dense model near one billion parameters, compare SwiGLU, reduced-branch TriGLU, polynomial activations, and factorized HyperGrid variants under matched FLOPs, and only then scale to MoE or recurrent architectures.

9.4 Scaling perspective

The long-term motivation is not that EE removes attention cost. Full attention and KV-cache requirements remain major constraints, and sparse or linear attention may only replace part of a production model's dense attention. The claim is narrower: for a fixed attention burden, richer FFN interactions can increase the model's representational options with compute and activation memory that scale linearly in sequence length. SwiGLU's success over simpler MLPs is one precedent; whether more aggressive EE mechanisms preserve a useful quality/throughput frontier at large scale remains an empirical question.

10 Conclusion

We proposed *Expressivity Engineering* as a practical lens for organizing multiplicative, higher-order, routed, and dynamically parameterized neural computation. Using the layer-localization and experimental workflow of Zhang et al. [2026] as guide rails, we tested one high-capacity EE module at Qwen3-1.7B-Base Layer 10 under matched single-layer GRPO. The intervention starts as an exact functional no-op and differs from the baseline only through the added side architecture and its parameters.

Across five matched checkpoints, the EE variant improves the primary Math average and the provisional Code average, with especially consistent gains on OlympiadBench, corrected LiveCodeBench, and strict GPQA-Diamond-Freeform. It does not uniformly dominate: aggregate Reasoning and Language are tied or slightly lower, MGSM contains a severe late outlier, and serving throughput is roughly half that of vanilla Qwen under the measured setup. The appropriate conclusion is therefore a positive, artifact-backed pilot: added FFN expressivity can improve a strong single-layer RL baseline on several hard tasks, but the result still requires multi-seed replication, protocol closure, precision ablation, and efficiency engineering before it can support a broad claim.

Artifact Availability

The accompanying repository contains architecture code, configs, deterministic data-order contracts, evaluation scripts, compact runtime receipts, manual audit packages, and the corrected consolidated result table. Model weights, datasets, checkpoints, uncurated generations, private operational logs, and chat transcripts are excluded from the research release. The report's numeric source of truth is the repository record `docs/experiment_records/2026-07-18_qwen3-1p7b-math-ood-corrected-consolidated-master-table.md`.

References

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021. URL https://arxiv.org/abs/2108.07732.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL https://arxiv.org/abs/2110.14168.

David Ha, Andrew Dai, and Quoc V. Le. Hypernetworks, 2016. URL https://arxiv.org/abs/1609.09106.

Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Leng Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, Jie Liu, Lei Qi, Zhiyuan Liu, and Maosong Sun. Olympiadbench: A challenging benchmark for promoting agi with olympiad-level bilingual multimodal scientific problems, 2024. URL https://arxiv.org/abs/2402.14008.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. In *Advances in Neural Information Processing Systems*, 2021. URL https://arxiv.org/abs/2103.03874.

Yuzhen Huang, Yuzhuo Bai, Zhihao Zhu, Junlei Zhang, Jinghan Zhang, Tangjun Su, Junteng Liu, Chuancheng Lv, Yikai Zhang, Yao Fu, Maosong Sun, and Zhiyuan Liu. C-eval: A multi-level multi-discipline chinese evaluation suite for foundation models, 2023. URL https://arxiv.org/abs/2305.08322.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code, 2024. URL https://arxiv.org/abs/2403.07974.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention, 2023. URL https://arxiv.org/abs/2309.06180.

Jia Li, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Costa Huang, Kashif Rasul, Longhui Yu, Albert Jiang, Ziju Shen, Zihan Qin, Bin Dong, Li Zhou, Yann Fleureau, Guillaume Lample, and Stanislas Polu. Numinamath. Hugging Face dataset repository, 2024. URL https://huggingface.co/AI-MO/NuminaMath-CoT.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems*, 2023. URL https://arxiv.org/abs/2305.01210.

Chuancheng Lv, Lei Li, Shitou Zhang, Gang Chen, Fanchao Qi, Ningyu Zhang, and Hai-Tao Zheng. Hyperlora: Efficient cross-task generalization via constrained low-rank adapters generation. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 16376–16393, 2024. doi: 10.18653/v1/2024.findings-emnlp.956. URL https://aclanthology.org/2024.findings-emnlp.956/.

Rabeeh Karimi Mahabadi, Sebastian Ruder, Mostafa Dehghani, and James Henderson. Parameter-efficient multi-task fine-tuning for transformers via shared hypernetworks. In *Proceedings of ACL-IJCNLP*, pages 565–576, 2021. doi: 10.18653/v1/2021.acl-long.47. URL https://aclanthology.org/2021.acl-long.47/.

Mathematical Association of America. American mathematics competitions: Amc 10 and amc 12. Official competition information, 2023. URL https://maa.org/student-programs/amc/.

Zeju Qiu, Weiyang Liu, Haiwen Feng, Yuxuan Xue, Yao Feng, Zhen Liu, Dan Zhang, Adrian Weller, and Bernhard Schölkopf. Controlling text-to-image diffusion by orthogonal finetuning. In *Advances in Neural Information Processing Systems*, 2023. URL https://arxiv.org/abs/2306.07280.

David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. Gpqa: A graduate-level google-proof q&a benchmark, 2023. URL https://arxiv.org/abs/2311.12022.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL https://arxiv.org/abs/2402.03300.

Noam Shazeer. Glu variants improve transformer, 2020. URL https://arxiv.org/abs/2002.05202.

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 1279–1297, 2025. doi: 10.1145/3689031.3696075.

Freda Shi, Mirac Suzgun, Markus Freitag, Xuezhi Wang, Suraj Srivats, Soroush Vosoughi, Hyung Won Chung, Yi Tay, Sebastian Ruder, Denny Zhou, Dipanjan Das, and Jason Wei. Language models are multilingual chain-of-thought reasoners, 2022. URL https://arxiv.org/abs/2210.03057.

Yi Tay, Zhe Zhao, Dara Bahri, Donald Metzler, and Da-Cheng Juan. Hypergrid: Efficient multi-task transformers with grid-wise decomposable hyper projections, 2020. URL https://arxiv.org/abs/2007.05891.

Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhu Chen. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark, 2024. URL https://arxiv.org/abs/2406.01574.

An Yang et al. Qwen3 technical report, 2025. URL https://arxiv.org/abs/2505.09388.

Ted Zadouri, Ahmet Üstün, Arash Ahmadian, Beyza Ermis, Acyr Locatelli, and Sara Hooker. Pushing mixture of experts to the limit: Extremely parameter efficient moe for instruction tuning. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/6d00071564ec447466fc4577743cf1b3-Abstract-Conference.html.

Zijian Zhang, Rizhen Hu, Athanasios Glentis, Dawei Li, Chung-Yiu Yau, Hongzhou Lin, and Mingyi Hong. Is one layer enough? training a single transformer layer can match full-parameter rl training, 2026. URL https://arxiv.org/abs/2607.01232.

Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models, 2023. URL https://arxiv.org/abs/2311.07911.

Zhijian Zhuo, Ya Wang, Yutao Zeng, Xiaoqing Li, Xun Zhou, and Jinwen Ma. Polynomial composition activations: Unleashing the dynamics of large language models, 2024. URL https://arxiv.org/abs/2411.03884.